

Crear RDS, crear la base de datos

Elegir un método de creación de base de datos

☐ Configuración completa
Puede definir todos los aspectos de configuración, incluidas las de disponibilidad, seguridad, copias de seguridad y mantenimiento.

☒ Creación sencilla
Utilice las configuraciones recomendadas. Algunas opciones de configuración se pueden cambiar después de crear la base de datos.

Configuración

Tipo de motor [Información](#)

☐ Aurora (MySQL Compatible) 

☐ Aurora (PostgreSQL Compatible) 

☐ MySQL 

☐ PostgreSQL 

☒ MariaDB 

☐ Oracle 

☐ Microsoft SQL Server 

☐ IBM Db2 

Tamaño de la instancia de base de datos

☐ Producción
db.r7g.xlarge
4 vCPUs
32 GiB RAM
400 GiB

☐ Desarrollo y pruebas
db.r7g.large
2 vCPUs
16 GiB RAM
200 GiB

☒ Entorno de pruebas
db.r7g.micro
2 vCPUs
1 GiB RAM
20 GiB

Identificador de instancias de bases de datos

Escriba un nombre para la instancia de base de datos. El nombre debe ser único en relación con todas las instancias de base de datos pertenecientes a su cuenta de AWS en la región de AWS actual.

☒ El identificador de instancias de bases de datos ya existe

El identificador de la instancia de base de datos no distingue entre mayúsculas y minúsculas, pero se almacena con todas las letras en minúsculas (como en "mydbinstance"). Restricciones: de 1 a 63 caracteres alfanuméricos o guiones. El primer carácter debe ser una letra. No puede contener dos guiones consecutivos. No puede terminar con un guion.

Nombre de usuario maestro [Información](#)

Escriba un ID de inicio de sesión para el usuario maestro de la instancia de base de datos.

1 a 16 caracteres alfanuméricos. El primer carácter debe ser una letra.

Administración de credenciales

Puede usar AWS Secrets Manager o administrar sus credenciales de usuario maestro.

☐ Administrado en AWS Secrets Manager - más seguro
RDS genera una contraseña y la administra durante todo su ciclo de vida mediante AWS Secrets Manager.

☒ Autoadministrado
Cree su propia contraseña o pida a RDS que cree una contraseña para que pueda administrarla.

☐ Generar contraseña automáticamente
Amazon RDS puede generar una contraseña en su nombre, o bien puede especificar su propia contraseña.

Contraseña maestra [Información](#)

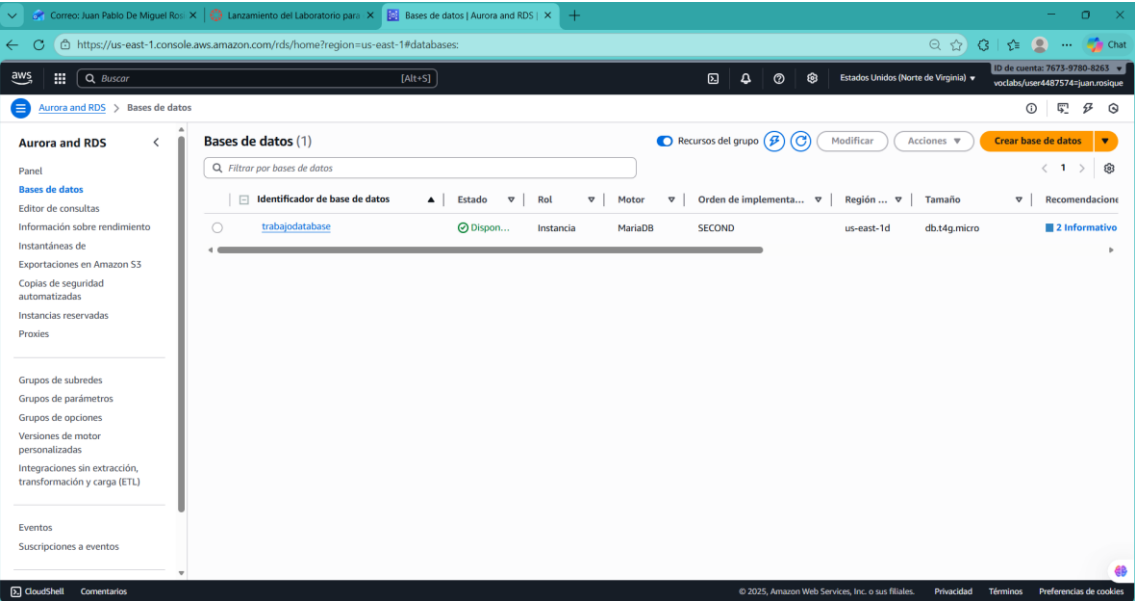
Password strength **Strong**

Restricciones mínimas: al menos 8 caracteres ASCII imprimibles. No puede contener ninguno de los siguientes símbolos: / * " @

Confirmar la contraseña maestra [Información](#)

(el nombre me sale el rojo, porque voy a hacer la guía a partir de una base de datos ya creada, el admin lo dejas tal cual (te viene por defecto), tienes que poner la contraseña y autoadministración ya te viene por defecto)

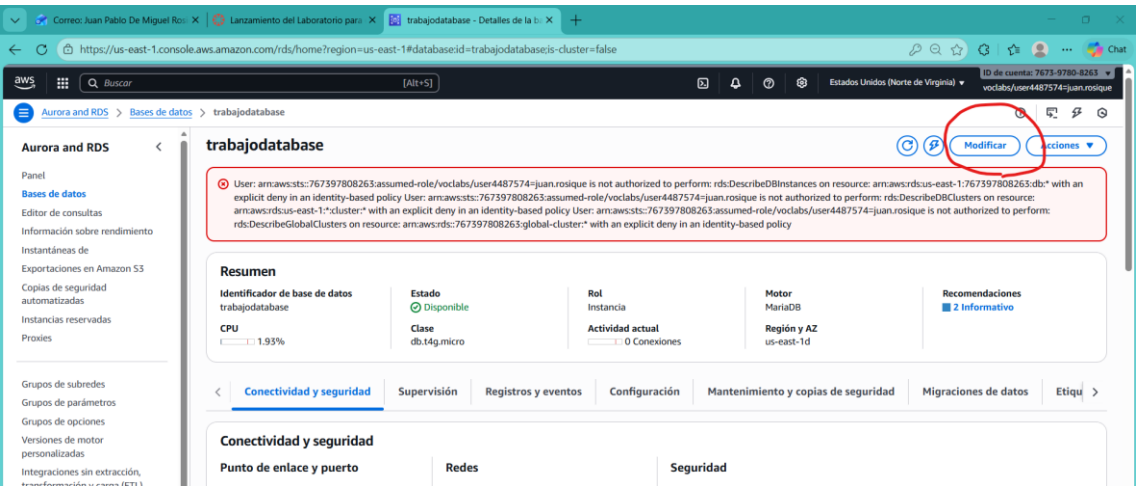
Hay que esperar unos minutillos a que se ponga disponible, y posteriormente entrar al nombre de la base de datos y subrayado.



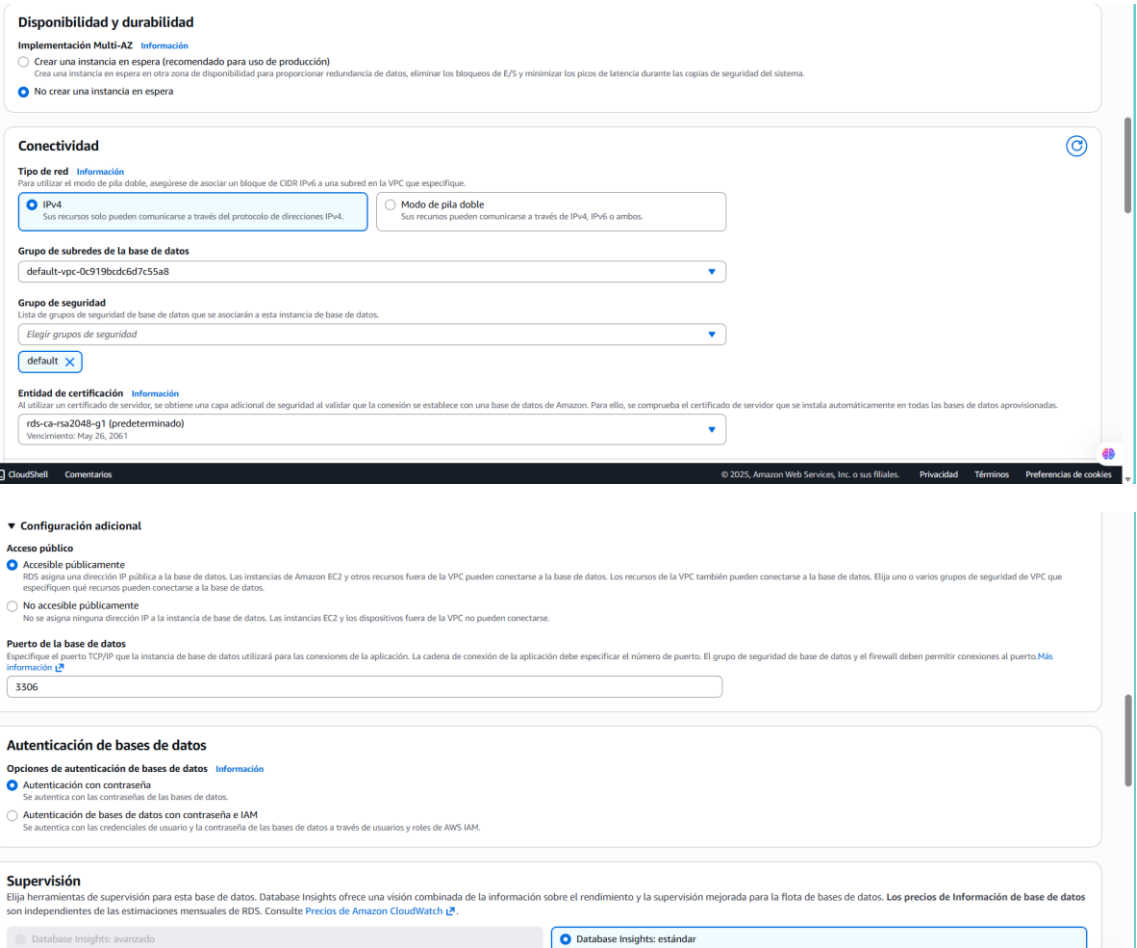
Vale pero ahora no es accesible, vamos a abrirla



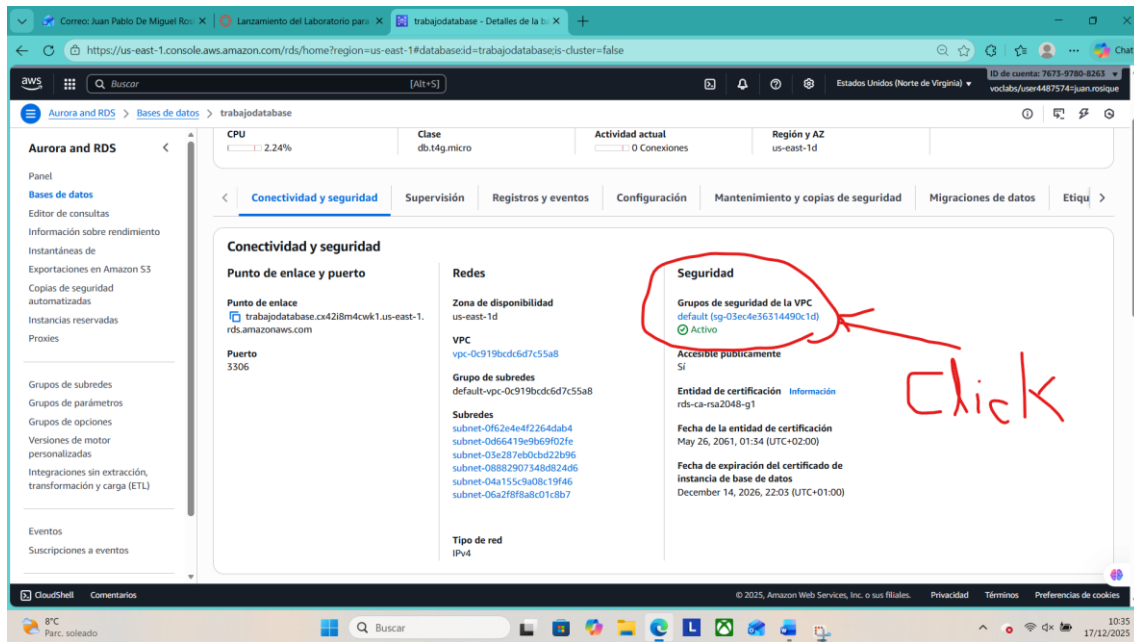
Vamos a conectividad y seguridad (modificamos la base de datos)



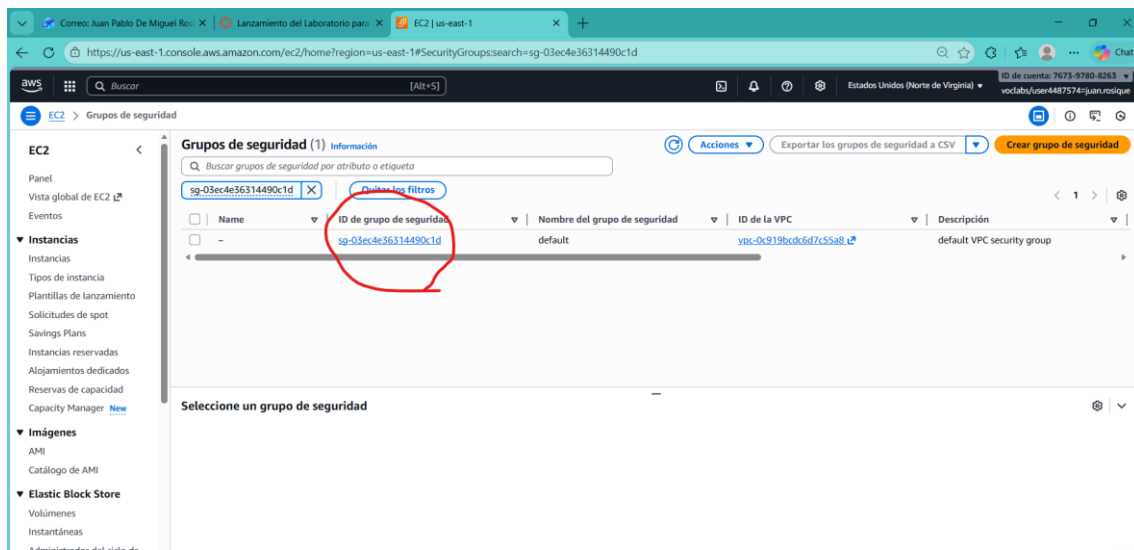
Ahora tras hacer lo anterior, dejamos esta configuración



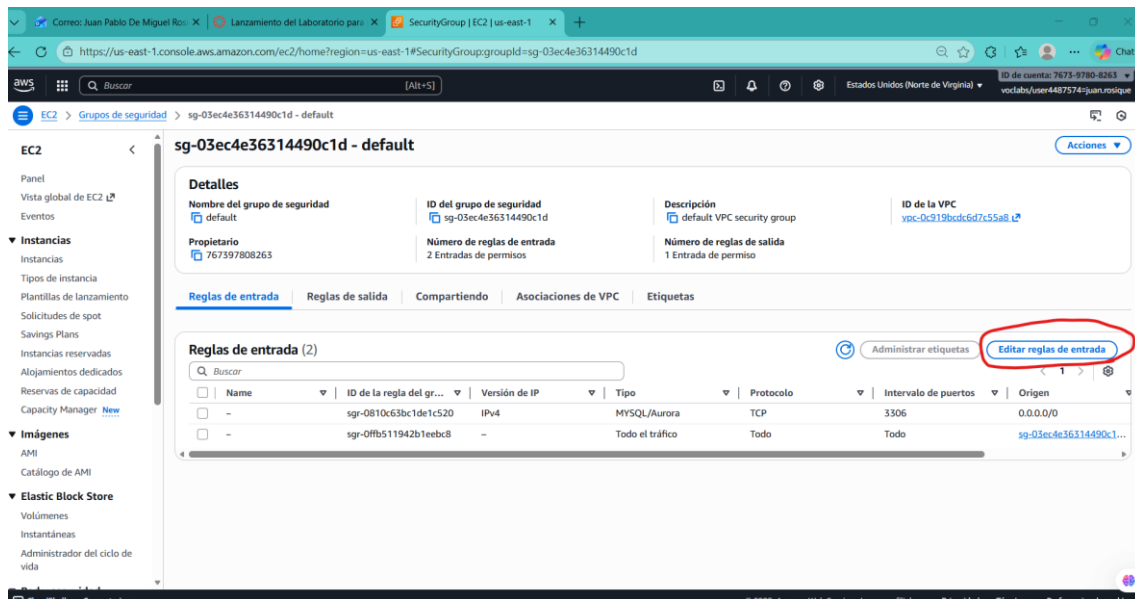
Ahora vamos a grupos de seguridad y hacemos click



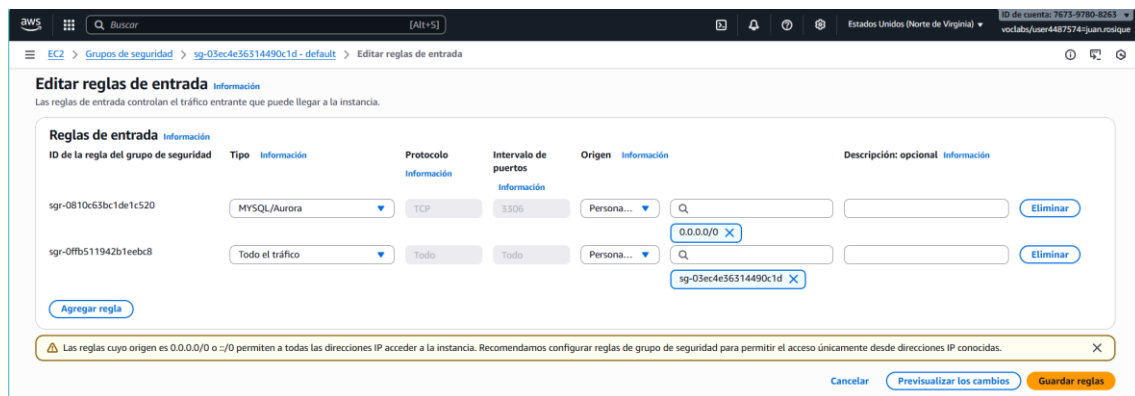
Volvemos a dar click



y le damos a editar reglas de entrada



Verificamos que tengamos esto, si es así tiramos pal ante ino hay que dejarlo así



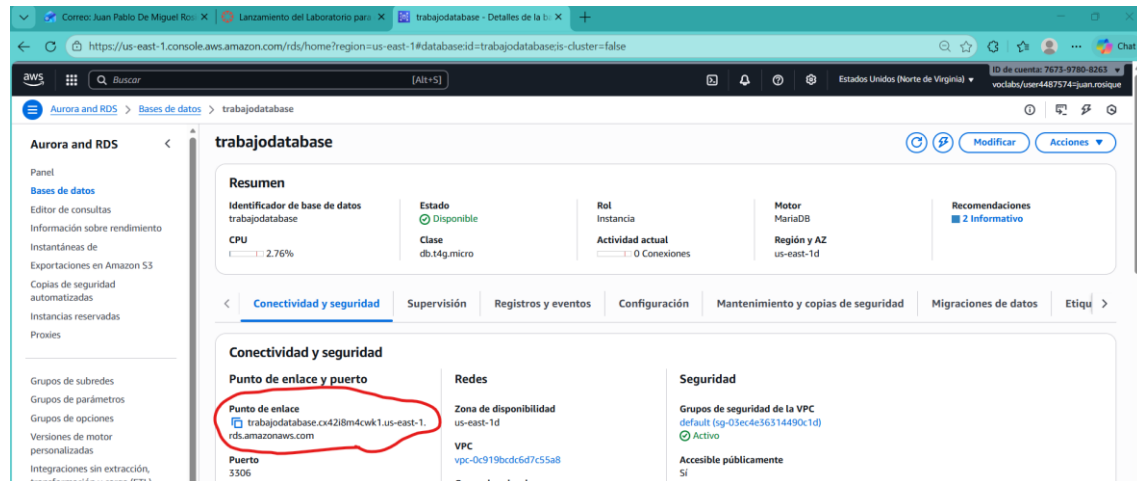
Nota importante, de cada vez que hacemos cambios en la configuración de rds, tarda unos minuuios, 5-10 noo desesperarse

Nos conectamos a la base de datos y la dejamos preparada

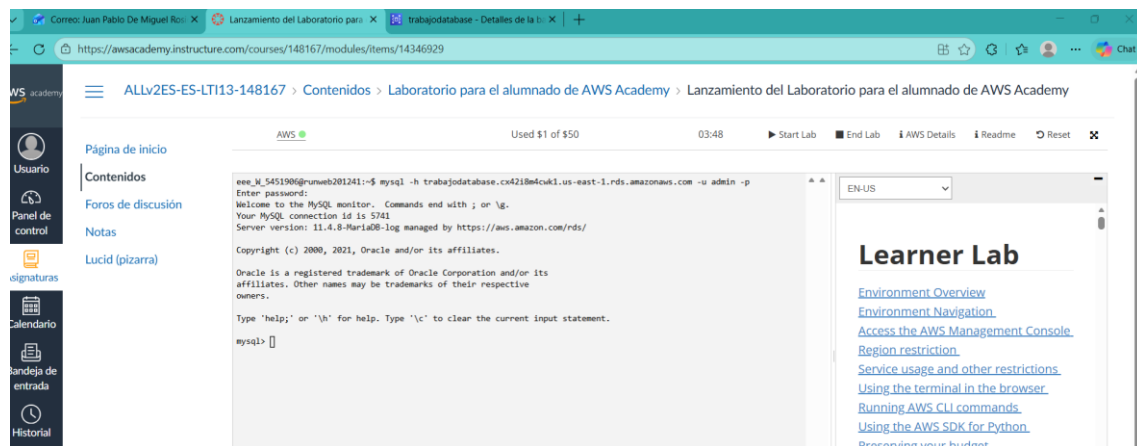
Vamos a utilizar la instancia RDS que creamos en prácticas anteriores (por ej. calculadora). Necesitamos crear una tabla para guardar las opiniones.

1. Conéctate a tu base de datos (usando mysql desde CloudShell o tu cliente preferido). Por ej.

Necesitas copiar lo siguiente:



mysql -h trabajodatabase.cx42i8m4cw1.us-east-1.rds.amazonaws.com -u admin -p



(te debería salir eso)

2. Tras lo anterior, en el mismo lugar, Ejecutas el siguiente script SQL para crear la tabla:

CREATE DATABASE trabajodatabase;

```

AWS ● Used $1 of $50 03:46 ▶ Start Lab ■ End Lab
inicio
;
scusión
ra)

eee_W_5451906@runweb201241:~$ mysql -h trabajodatabase.cx42i8m4cwk1.us-east-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 5741
Server version: 11.4.8-MariaDB-log managed by https://aws.amazon.com/rds/

Copyright (c) 2000, 2021, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE trabajodatabase;
ERROR 1007 (HY000): Can't create database 'trabajodatabase'; database exists
mysql> 
```

Posteriormente, ejecuta lo siguiente:

```
USE trabajodatabase; /* Asegúrate de usar tu nombre de DB correcta */
CREATE TABLE reviews (
  id INT AUTO_INCREMENT PRIMARY KEY,
  video_filename VARCHAR(255) NOT NULL,
  user_name VARCHAR(100) NOT NULL,
  comment TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```

mysql> USE ESTUDIO; /* Asegúrate de usar tu nombre de DB correcta */
Database changed
mysql> CREATE TABLE reviews (
-> id INT AUTO_INCREMENT PRIMARY KEY,
-> video_filename VARCHAR(255) NOT NULL,
-> user_name VARCHAR(100) NOT NULL,
-> comment TEXT NOT NULL,
-> created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
-> );
Query OK, 0 rows affected (0.07 sec)

mysql> 
```

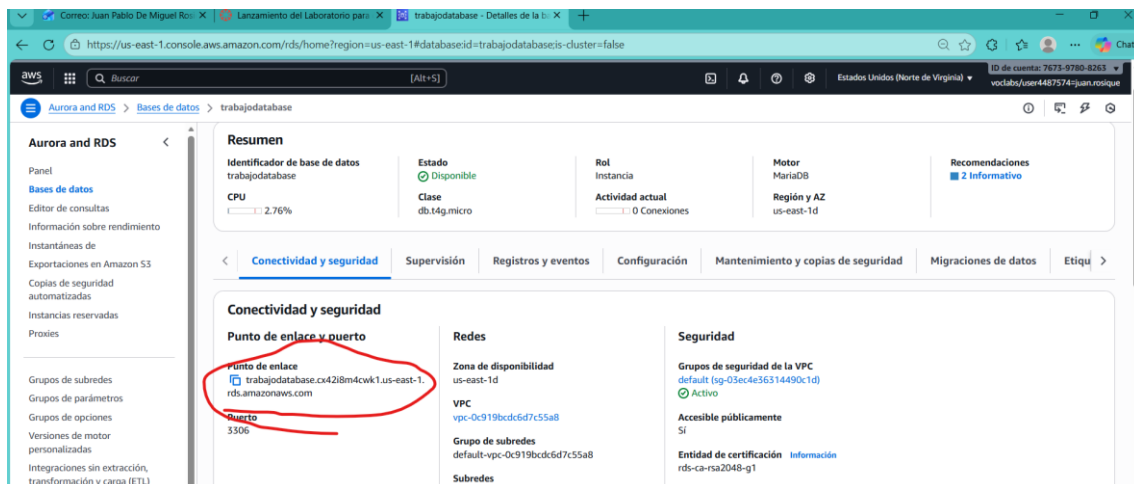
(esto es lo que debería salirte)

Ahora comenzamos a editar los archivos lambda, es importante decir que la estructura cambia según el caso por ello tenemos que meternos a ver individualmente cada una para saber que poner

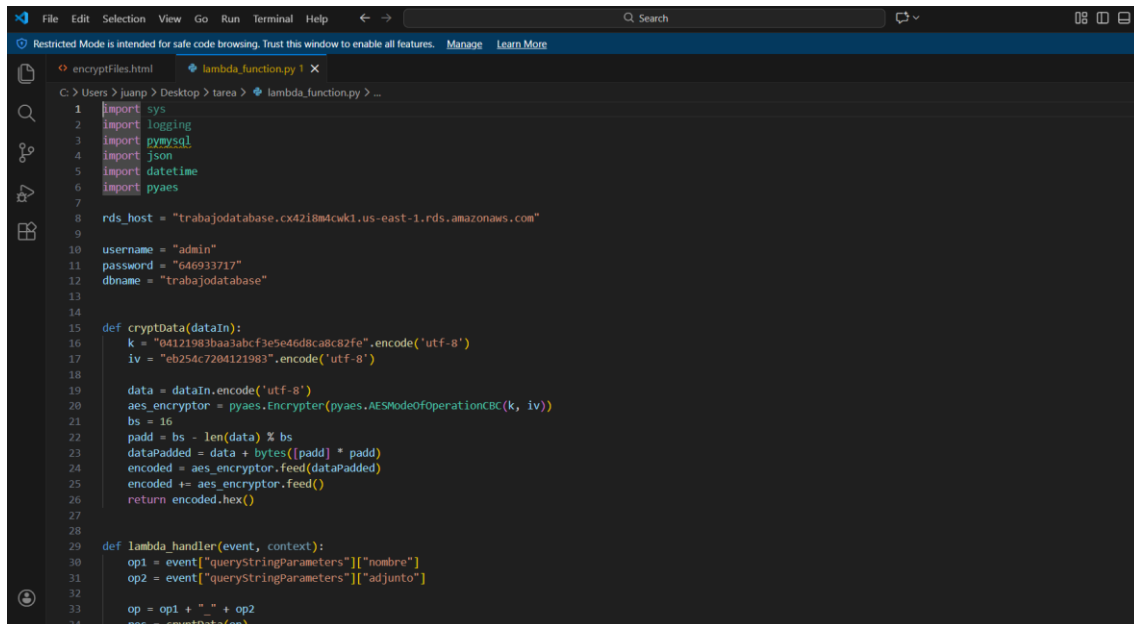
En este archivo lamda-function, debes cambiar lo que se ve acontinuacion(solo rds_host, username, password, dbname)(en mi caso seria)(enlace de puerto, admin(es como le llame), password que le pusiste a la base de datos, trabajodatabase))

```
1 import sys
2 import logging
3 import pymysql
4 import json
5 import datetime
6 import pyaes
7
8 rds_host = "3.219.58.166" #esto es lo que tenemos que editar para nuestro caso
9
10 username = "ejemplo"
11 password = "114054"
12 dbname = "alejandro_zabaleta" # hasta aqui
13
14
15 def cryptData(dataIn):
16     k = "04121981Baa3abcF1e5e46d8ca8c82fe".encode('utf-8')
17     iv = "eb254c7204121981".encode('utf-8')
18
19     data = dataIn.encode('utf-8')
20     aes_encryptor = pyaes.Encrypter(pyaes.AESModeOfOperationCBC(k, iv))
21     bs = 16
22     padd = bs - len(data) % bs;
23     dataPadded = data + bytes([padd] * padd);
24     encoded = aes_encryptor.feed(dataPadded);
25     encoded += aes_encryptor.feed();
26     return encoded.hex()
27
28
29 def lambda_handler(event, context):
30     op1=event["queryStringParameters"]["nombre"];
31     op2=event["queryStringParameters"]["adjunto"];
32
33     op=op1+"-"+op2;
34     res=cryptData(op);
35
36     try:
37         conn = pymysql.connect(rds_host, user=username, passwd=password, db=dbname, connect_timeout=10, port=3306)
38         with conn.cursor() as cur:
39             cur.execute("insert into cifrado values ('"+op+"','"+res+"','"+datetime.datetime.utcnow().strftime('%Y/%m/%d %H:%M:%S')+ "')");
40             conn.commit();
41             cur.close();
42     except pymysql.MySQLError as e:
43         print (e);
44         conn.close();
45
46     return {
47         'statusCode': 200,
48         'headers': { 'Access-Control-Allow-Origin': '*' },
49         'body': json.dumps( { 'res': res } )
50     }
51
52
53
```

Rds_host: es el enlca lo sacamos de aquí



En mi caso quedaría algo como esto:

A screenshot of a code editor window with a dark theme. The window title is "lambda_function.py 1 X". The code is a Python lambda function that imports sys, logging, pymysql, json, datetime, and pyaes. It defines database connection parameters (rds_host, username, password, dbname) and a cryptData function that uses AES encryption. The lambda_handler function takes an event and context, extracts 'nombre' and 'adjunto' from the query string parameters, concatenates them, and passes the result to cryptData. The editor has a sidebar with icons for Explorer, Search, Source Control, and Run and Debug. The top bar shows "Restricted Mode is intended for safe code browsing. Trust this window to enable all features. Manage Learn More".

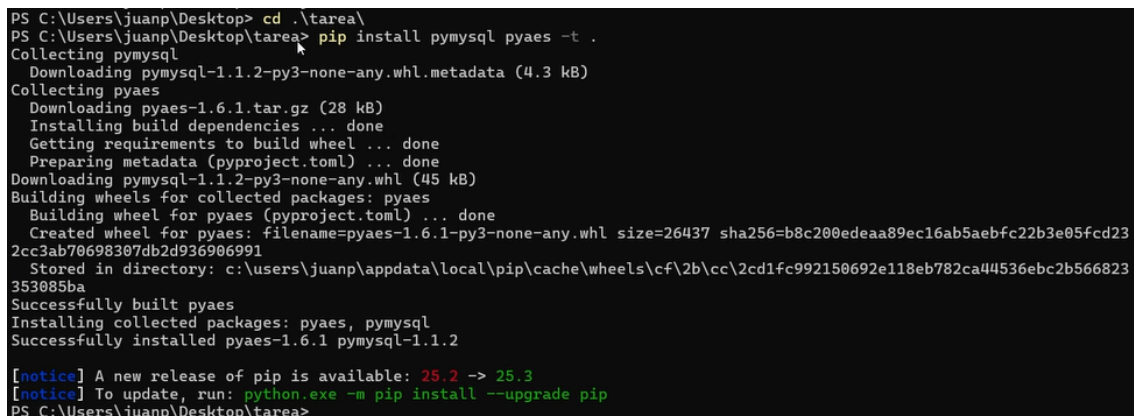
```
1 import sys
2 import logging
3 import pymysql
4 import json
5 import datetime
6 import pyaes
7
8 rds_host = "trabajodatabase.cx42i8m4cwk1.us-east-1.rds.amazonaws.com"
9
10 username = "admin"
11 password = "646933717"
12 dbname = "trabajodatabase"
13
14
15 def cryptData(dataIn):
16     k = "04121983baa3abcf3e5e46d8ca8c82fe".encode('utf-8')
17     iv = "eb254c7204121983".encode('utf-8')
18
19     data = dataIn.encode('utf-8')
20     aes_encryptor = pyaes.Encrypter(pyaes.AESModeOfOperationCBC(k, iv))
21     bs = 16
22     padd = bs - len(data) % bs
23     dataPadded = data + bytes([padd] * padd)
24     encoded = aes_encryptor.feed(dataPadded)
25     encoded += aes_encryptor.feed()
26     return encoded.hex()
27
28
29 def lambda_handler(event, context):
30     op1 = event["queryStringParameters"]["nombre"]
31     op2 = event["queryStringParameters"]["adjunto"]
32
33     op = op1 + "_" + op2
34     res = cryptData(op)
```

Es posible que tengamos una o mas funciones lambda, realmente es ir metiendo los anclajes y puntos de acceso donde corresponda

Una vez editado hay que preparar el zip para la subida, ejecutaremos este comando para instalar las dependencias

Entras en el directorio donde tengas tu lambda, y ejecutas el comando:

Pip install pymysql pyaes -t .

A screenshot of a Windows command prompt window. The prompt shows the execution of 'pip install pymysql pyaes -t .' in the directory 'C:\Users\juarp\Desktop\tarea'. The output shows the collection and installation of pymysql-1.1.2 and pyaes-1.6.1. The prompt ends with a notice about a new release of pip (25.2 -> 25.3) and instructions to update it.

```
PS C:\Users\juarp\Desktop> cd .\tarea\
PS C:\Users\juarp\Desktop\tarea> pip install pymysql pyaes -t .
Collecting pymysql
  Downloading pymysql-1.1.2-py3-none-any.whl.metadata (4.3 kB)
Collecting pyaes
  Downloading pyaes-1.6.1.tar.gz (28 kB)
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Downloaded pymysql-1.1.2-py3-none-any.whl (45 kB)
Building wheels for collected packages: pyaes
  Building wheel for pyaes (pyproject.toml) ... done
  Created wheel for pyaes: filename=pyaes-1.6.1-py3-none-any.whl size=26437 sha256=b8c200edaaa89ec16ab5aebfc22b3e05fcd232cc3ab70698307db2d936906991
  Stored in directory: c:\users\juarp\appdata\local\pip\cache\wheels\cf\2b\cc\2cd1fc992150692e118eb782ca44536ebc2b566823353085ba
Successfully built pyaes
Installing collected packages: pyaes, pymysql
Successfully installed pyaes-1.6.1 pymysql-1.1.2

[notice] A new release of pip is available: 25.2 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\juarp\Desktop\tarea>
```

Dependerá del arvhco concreto

Lo metemos en un zip el .py

Es un .py por cada función lambda

Bucamos funciones lambda

Para ello, en el buscador superior de arriba, en vez de rds, buscamos lambda, nos metemos en funciones, donde veremos las lambdas que tenemos, para este caso, vamos a crear dos lambdas.

Funciones (7)

Última obtención hace 0 segundos

Acciones

Crear función

Q Search by attributes or search by keyword

< 1 >

☐	Nombre de la función	Descripción	Tipo de paquete	Tiempo de ejecución	Tipo	Última modificación
☐	RoleCreationFunction	Create SLR if absent	Zip	Python 3.9	Estándar	hace 2 semanas
☐	lambdaFunction	-	Zip	Python 3.12	Estándar	hace 2 días
☐	MainMonitoringFunction	-	Zip	Python 3.9	Estándar	hace 2 semanas
☐	ModLabRole	updates LabRole to allow it to assume itself	Zip	Python 3.9	Estándar	hace 2 semanas
☐	lambaKeys2	-	Zip	Python 3.12	Estándar	hace 2 días
☐	RedshiftOverwatch	Deletes Redshift Cluster if the count is more than 2.	Zip	Python 3.9	Estándar	hace 2 semanas
☐	RedshiftEventSubscription	Create Redshift event subscription to SNS Topic.	Zip	Python 3.9	Estándar	hace 2 semanas

Le damos a crear

Pondremos esta configuración

Lambda > Funciones > Crear función

Crear función

Información

Seleccione una de las siguientes opciones para crear la función.

Crear desde cero

Empiece con un sencillo ejemplo "Hello World".

Usar un esquema

Cree una aplicación Lambda usando un código de muestra y los ajustes de configuración predefinidos de casos de uso comunes.

Imagen del contenedor

Seleccione una imagen de contenedor para implementar para la función.

Información básica

Nombre de la función

Ingrese un nombre para describir el propósito de la función.

LambdaEncrypt

El nombre de la función debe tener entre 1 y 64 caracteres, debe ser exclusivo de la región y no puede incluir espacios. Los caracteres válidos son a-z, A-Z, 0-9, guiones (-) y guiones bajos (_).

Versión ejecutable

Información

Elige el lenguaje que desea usar para escribir la función. Tenga en cuenta que el editor de código de la consola solo admite Node.js, Python y Ruby.

Python 3.12

Durable execution - new

Información

Enable durable execution to simplify building resilient multi-step applications that checkpoint progress and resume after interruptions. Supports Python and Node.js runtimes. [View pricing](#).

Activar

Arquitectura

Información

Elige la arquitectura del conjunto de instrucciones que desea para el código de la función.

arm64

x86_64

Permisos

Información

De forma predeterminada, Lambda creará un rol de ejecución con permisos para cargar registros en Amazon CloudWatch Logs. Puede personalizar este rol predeterminado más adelante al agregar los disparadores.

▼ Cambiar el rol de ejecución predeterminado

Rol de ejecución

Seleccione un rol que defina los permisos de la función. Para crear un rol personalizado, vaya a [la consola de IAM](#).

Creación de un nuevo rol con permisos básicos de Lambda

Uso de un rol existente

Creación de un nuevo rol desde la política de AWS templates

Rol existente

Seleccione un rol existente que haya creado para usarlo con esta función de Lambda. El rol debe tener permiso para cargar registros en Amazon CloudWatch Logs.

LabRole

[Consulte el rol LabRole](#) en la consola de IAM.

▼ Configuraciones adicionales

Use configuraciones adicionales para configurar las redes, la seguridad y la gobernanza de su función. Estos ajustes permiten proteger y personalizar la implementación de las funciones de Lambda.

Tipo de computación

Seleccione un tipo de procesamiento: sin servidor con precios de pago por uso y escalamiento automático, o escalamiento mejorado en instancias de EC2 con capacidad dedicada y precios diferentes. Esta configuración no se puede cambiar después de crear la función.

Lambda (predeterminado)

Amplía y pone en marcha código automáticamente sin necesidad de administrar servidores. Pague solo por el tiempo de procesamiento que use.

Instancias administradas de Lambda: novedad

Ejecute en instancias de EC2 administradas por Lambda con escalado de procesamiento mejorado. Debe crear antes un proveedor de capacidad para usar esta opción. Se aplica un modelo de precios diferente en función del uso de EC2. [Consulte los precios](#).

Activar Windows

Ve a Configuración para activar Windows.

▼ Configuraciones adicionales

Use configuraciones adicionales para configurar las redes, la seguridad y la gobernanza de su función. Estos ajustes permiten proteger y personalizar la implementación de las funciones de Lambda.

Tipo de computación

Seleccione un tipo de procesamiento: sin servidor con precios de pago por uso y escalamiento automático, o escalamiento mejorado en instancias de EC2 con capacidad dedicada y precios diferentes. Esta configuración no se puede cambiar después de crear la función.

Lambda (predeterminado)

Amplía y pone en marcha código automáticamente sin necesidad de administrar servidores. Pague solo por el tiempo de procesamiento que use.

Instancias administradas de Lambda: novedad

Ejecute en instancias de EC2 administradas por Lambda con escalado de procesamiento mejorado. Debe crear antes un proveedor de capacidad para usar esta opción. Se aplica un modelo de precios diferente en función del uso de EC2. [Consulte los precios](#).

Redes

URL de función

Información

Use URL de función para asignar puntos de enlace HTTP(S) a la función de Lambda.

Activar

Tipo de autorización

Elige el tipo de autorización para la URL de la función. [Más información](#)

AWS_IAM

Solo los usuarios y roles de IAM autenticados pueden realizar solicitudes a la URL de la función.

NONE

Lambda no realizará la autenticación de IAM en las solicitudes a la URL de la función. El punto de conexión de la URL será público a menos que implemente su propia lógica de autorización en la función.

Permisos de URL de función

ⓘ Cuando elige el tipo de autenticación NONE, Lambda crea automáticamente la siguiente política basada en recursos y la asocia a la función. Esta política hace que la función sea pública para cualquier persona que tenga la URL de la función. Puede editar la política más adelante. Para limitar el acceso a los usuarios y roles de IAM autenticados, elija el tipo de autenticación AWS_IAM.

▼ Instrucción de política

```
1 * {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "StatementId": "FunctionURLAllowPublicAccess",
6       "Effect": "Allow",
7       "Principal": "*",
8       "Action": "lambda:InvokeFunctionUrl",
9       "Resource": "arn:aws:lambda:us-east-1:500183824007:function:LambdaEncrypt",
10      "Condition": {
11        "StringEquals": {
12          "lambda:FunctionUrlAuthType": "NONE"
13        }
14      }
15    },
16    {
17      "StatementId": "FunctionURLAllowInvokeAction",
18      "Effect": "Allow",
19      "Principal": "*",
20      "Action": "lambda:InvokeFunction",
21      "Resource": "arn:aws:lambda:us-east-1:500183824007:function:LambdaEncrypt",
22      "Condition": {
23        "Bool": {
24          "lambda:InvokedViaFunctionUrl": "true"
25        }
26      }
27    }
28  ]
29 }
```

Activar Windows

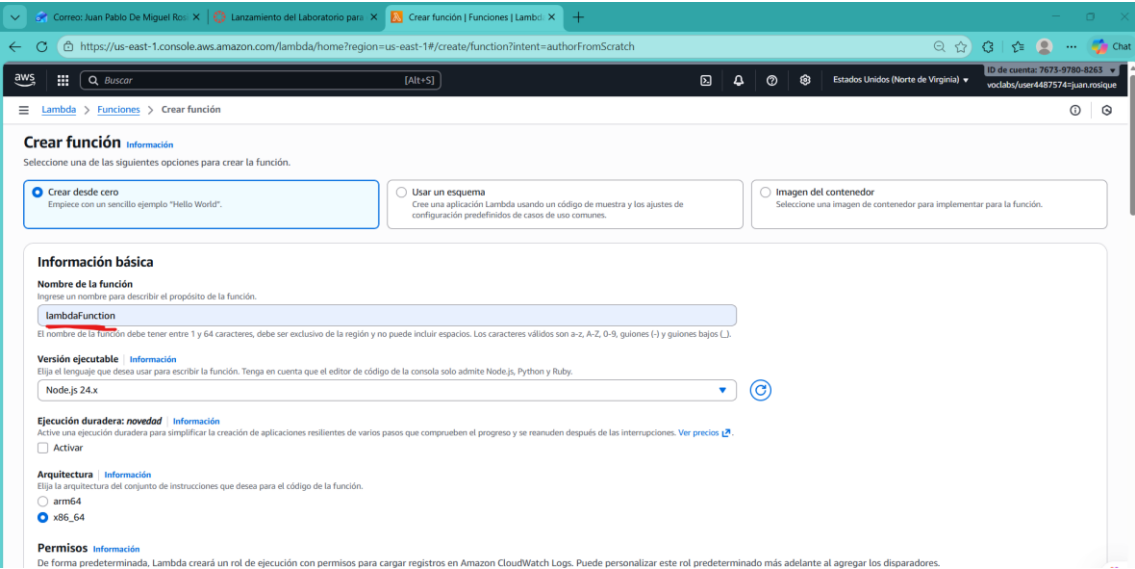
Ve a Configuración para activar Windows.

Modo de invocación

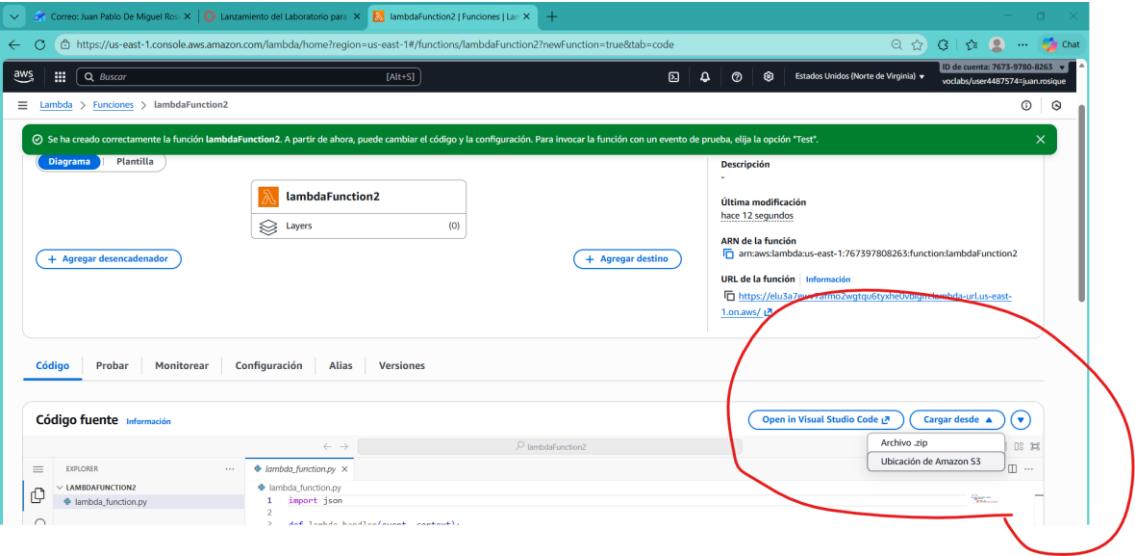
Información

1:1 JSON

En mi caso la llame lamdaFunction



La creamos, posteriormente cargamos el .zip en el que esta lambda function y donde ejecutamos el comando en la terminal.



pyaes	Carpeta de archivos		
pyaes-1.6.1.dist-info	Carpeta de archivos		
pymysql	Carpeta de archivos		
pymysql-1.1.2.dist-info	Carpeta de archivos		
lambda_function	Archivo de origen Python	1 KB	No

Y todo esto esta dentro de un zip:

pymysql	14/12/2025 23:43	Carpeta comprimi...	187 KB
---------	------------------	---------------------	--------

Cargar un archivo .zip

Al cargar un nuevo paquete de archivos .zip, se sobrescribe el código existente.

Cargar

pymysql.zip190.75 KB

Para los archivos mayores de 10 MB, considere la posibilidad de cargarlos usando Amazon S3.

Cifrado con una clave administrada por el cliente de AWS KMS

☐ De forma predeterminada, Lambda cifra el archivo .zip con una clave propiedad de AWS.

Cancelar

Guardar

Lo guardamos, y posteriormente, le daremos a deploy y test

Código fuente

Información

Open in Visual Studio Code

Cargar desde

EXPLORER

LAMBDAFUNCTION2

pyaes

pyaes-1.6.1.dist-info

pymysql

pymysql-1.1.2.dist-info

lambda_function.py

DEPLOY

Deploy (Ctrl+Shift+L)

Test (Ctrl+Shift+I)

TEST EVENTS (NONE SELECTED)

Create new test event

lambda_function.py

29

def lambda_handler(event, context):

37

try:

39

with conn.cursor() as cur:

40

cur.execute("insert into cifrado values ('" + res + "', " + datetime.datetime.utcnow().strftime("%Y/%m/%d %H:%M:%S") + ")")

41

conn.commit()

42

cur.close()

43

except pymysql.MySQLError as e:

44

print(e)

45

finally:

46

if conn:

47

conn.close()

48

return {

49

'statusCode': 200,

50

'headers': {'Access-Control-Allow-Origin': '*'},

51

'body': json.dumps({'res': res})

52

}

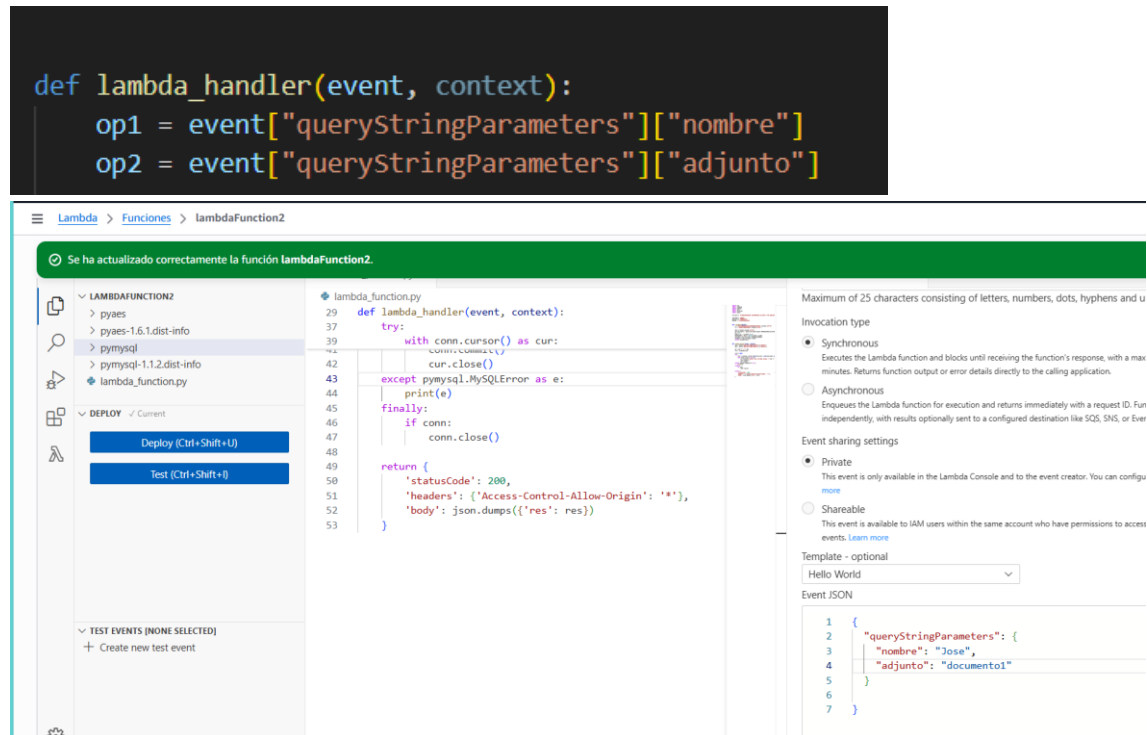
53

CloudShell

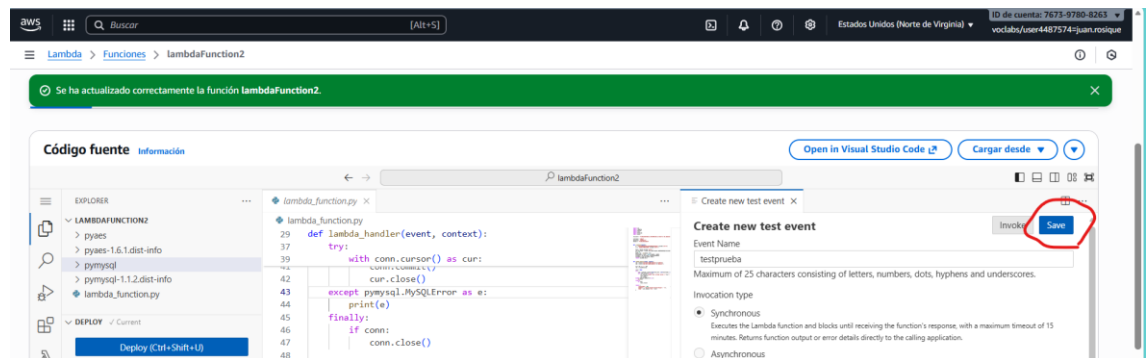
Comentarios

© 2025, Amazon Web Services, Inc. o sus filiales. Privacidad Términos Preferencias de cookies

Tras darle test, se nos abre una ventana a la derecha, no cambiamos nada, tan solo pon el nombre y debajo del todo, para hacer la query, usas lo que te viene en la lambda:



Tras rellenar esto, lo salvamos:

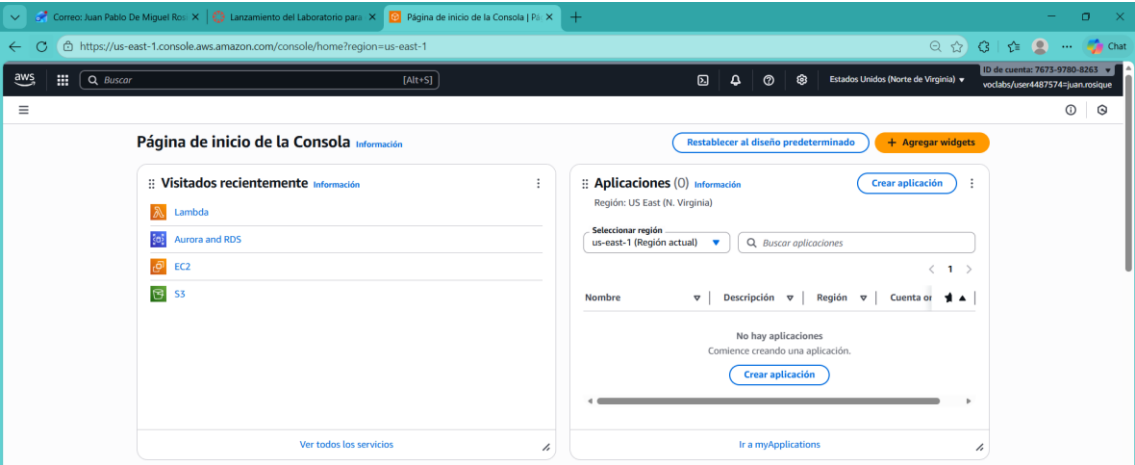


Después de hacer save, da de nuevo a deploy.

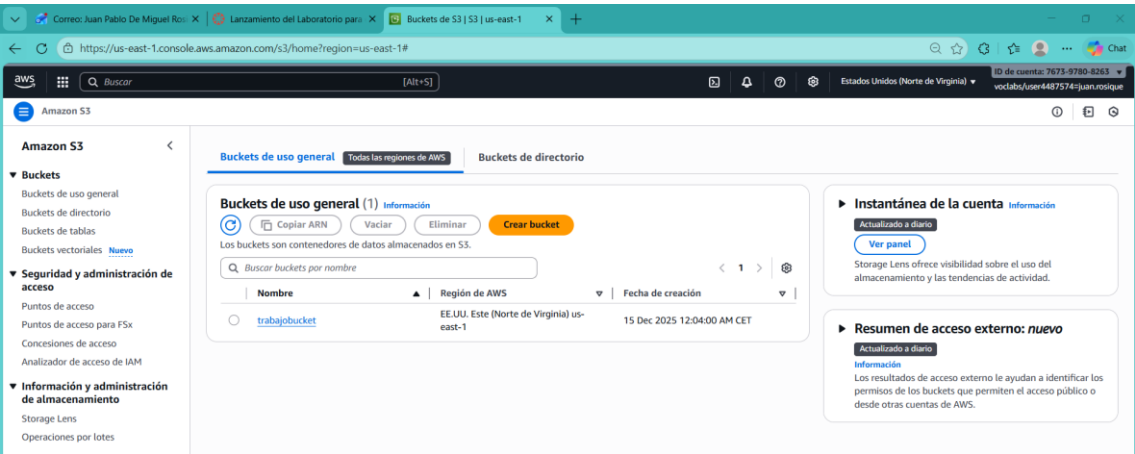
Importante es necesario subir y configurar igual que el otro el archivo .py para la función lambda que genere las keys,

Este es el código del archivo de claves, hay que adaptar el bucket y poner nuestro bucket

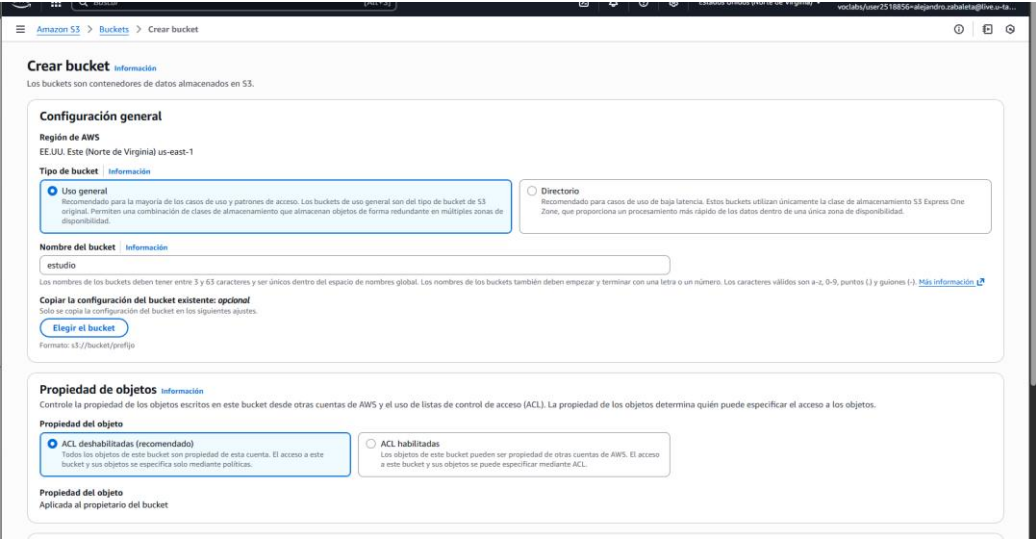
Ahora haremos el bucket para ello entramos en:



Buscamos bucket s3 y creamos uno



Creamos el bucket



Configuración de bloqueo de acceso público para este bucket

Se concede acceso público a los buckets y objetos a través de listas de control de acceso (ACL), políticas de bucket, políticas de puntos de acceso o todas las anteriores. A fin de garantizar que se bloquee el acceso público a todos sus buckets y objetos, active Bloquear todo el acceso público. Esta configuración se aplica exclusivamente a este bucket y a sus puntos de acceso. AWS recomienda activar Bloquear todo el acceso público, pero, antes de aplicar cualquiera de estos ajustes, asegúrese de que las aplicaciones funcionarán correctamente sin acceso público. Si necesita cierto nivel de acceso público a los buckets u objetos, puede personalizar la configuración individual a continuación para adaptarla a sus casos de uso de almacenamiento específicos. [Más información](#)

☐ **Bloquear todo el acceso público**
Activar esta configuración equivale a activar las cuatro opciones que aparecen a continuación. Cada uno de los siguientes ajustes son independientes entre sí.

- ☐ **Bloquear el acceso público a buckets y objetos concedido a través de nuevas listas de control de acceso (ACL)**
S3 bloqueará los permisos de acceso público aplicados a objetos o buckets agregados recientemente, y evitará la creación de nuevas ACL de acceso público para buckets y objetos existentes. Esta configuración no cambia los permisos existentes que permiten acceso público a los recursos de S3 mediante ACL.
- ☐ **Bloquear el acceso público a buckets y objetos concedido a través de cualquier lista de control de acceso (ACL)**
S3 ignorará todas las ACL que conceden acceso público a buckets y objetos.
- ☐ **Bloquear el acceso público a buckets y objetos concedido a través de políticas de bucket y puntos de acceso públicas nuevas**
S3 bloqueará las nuevas políticas de buckets y puntos de acceso que concedan acceso público a buckets y objetos. Esta configuración no afecta a las políticas ya existentes que permiten acceso público a los recursos de S3.
- ☐ **Bloquear el acceso público y entre cuentas a buckets y objetos concedido a través de cualquier política de bucket y puntos de acceso pública**
S3 ignorará el acceso público y entre cuentas en el caso de buckets o puntos de acceso que tengan políticas que concedan acceso público a buckets y objetos.

⚠ **Desactivar el bloqueo de todo acceso público puede provocar que este bucket y los objetos que contiene se vuelvan públicos**
AWS recomienda que active la opción para bloquear todo el acceso público, a menos que se requiera acceso público para casos de uso específicos y verificados, como el alojamiento de sitios web estáticos.

☒ Reconozco que la configuración actual puede provocar que este bucket y los objetos que contiene se vuelvan públicos.

○ [trabajobucket](#)

Cuando la crees, ve a permisos:

Objetos | Metadatos | Propiedades | **Permisos** | Métricas | Administración | Puntos de acceso

Información general sobre los permisos

Búsqueda de acceso
Los analizadores de acceso externos de IAM proporcionan los hallazgos de acceso. Obtenga más información sobre [cómo funcionan los hallazgos del analizador de IAM](#).
[Ver analizador para us-east-1](#)

Bloquear acceso público (configuración del bucket) [Editar](#)

Se concede acceso público a buckets y objetos a través de listas de control de acceso (ACL), políticas de bucket, políticas de puntos de acceso o todas las anteriores. A fin de garantizar que se bloquee el acceso público a todos sus buckets y objetos de S3, active Bloquear todo acceso público. Esta configuración se aplica en exclusiva a este bucket y a sus puntos de acceso. AWS recomienda activar Bloquear todo acceso público pero, antes de aplicar cualquiera de estos ajustes, asegúrese de que sus aplicaciones funcionarán correctamente sin acceso público. Si necesita cierto nivel de acceso público a sus buckets u objetos, puede personalizar los valores de configuración individuales a continuación para que se ajusten mejor a sus necesidades específicas de almacenamiento. [Más información](#)

Bloquear todo el acceso público
⚠ **Desactivado**
► Configuración de bloqueo de acceso público individual para este bucket

Política de bucket [Editar](#) [Eliminar](#)

La política del bucket, escrita en JSON, proporciona acceso a los objetos almacenados en el bucket. Las políticas de bucket no se aplican a los objetos que pertenecen a otras cuentas. [Más información](#)

✔ Política de buckets copiada

Y en policy, haz lo siguiente:

Política de bucket [Editar](#) [Eliminar](#)

La política del bucket, escrita en JSON, proporciona acceso a los objetos almacenados en el bucket. Las políticas de bucket no se aplican a los objetos que pertenecen a otras cuentas. [Más información](#)

✔ Política de buckets copiada

[Copiar](#)

```
{
  "Principal": "*",
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3::trabajobucket/*"
},
{
  "Sid": "PublicListBucket",
  "Effect": "Allow",
  "Principal": "*",
  "Action": "s3:ListBucket",
  "Resource": "arn:aws:s3::trabajobucket"
},
{
  "Sid": "PublicPutObject",
  "Effect": "Allow",
  "Principal": "*",
  "Action": "s3:PutObject",
  "Resource": "arn:aws:s3::trabajobucket/*"
}
}
```

Te dejo las políticas abajo, para que solo copies y pegues:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::trabajobucket/*"
    },
    {
      "Sid": "PublicListBucket",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::trabajobucket"
    },
    {
      "Sid": "PublicPutObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::trabajobucket/*"
    }
  ]
}
```

Una vez creado el bucket creamos una función lambda como la de antes con este código, lo adaptamos a nuestro bucket;

Literalmente la misma configuración lambda que el anterior, y aquí no cargamos el zip, debido a que el código estaba mal, copia y pega esto, en la parte del código, te mostrare, la parte donde se pega

```
import json, os, boto3, datetime, base64, hashlib, hmac
```

```
bucket = " database-bucket-TUBUCKET"
```

```
region = 'us-east-1'
```

```
def sign(key, msg):
```

```
    return hmac.new(key, msg.encode('utf-8'), hashlib.sha256).digest()
```

```
def getSignatureKey(key, dateStamp, regionName, serviceName):
```

```
    kDate = sign(('AWS4' + key).encode('utf-8'), dateStamp)
```

```
    kRegion = sign(kDate, regionName)
```

```
    kService = sign(kRegion, serviceName)
```

```
    kSigning = sign(kService, 'aws4_request')
```

```
    return kSigning
```

```
def lambda_handler(event, context):
```

```
    session = boto3.Session()
```

```
    creds = session.get_credentials().get_frozen_credentials()
```

```
    t = datetime.datetime.utcnow()
```

```
    amzDate = t.strftime('%Y%m%dT%H%M%SZ')
```

```
dateStamp = t.strftime('%Y%m%d')
```

```
policy_structure = {  
    "expiration": "2025-12-30T12:00:00.000Z",  
    "conditions": [  
        {"bucket": bucket},  
        ["starts-with", "$key", ""],  
        {"success_action_redirect": f"https://{bucket}.s3.us-east-  
1.amazonaws.com/success.html"},  
        {"x-amz-credential": creds.access_key + "/" + dateStamp + "/" + region +  
"/s3/aws4_request"},  
        {"x-amz-algorithm": "AWS4-HMAC-SHA256"},  
        {"x-amz-date": amzDate},  
        {"x-amz-security-token": creds.token}  
    ]  
}
```

```
policy_json = json.dumps(policy_structure)
```

```
stringToSign = base64.b64encode(policy_json.encode("utf-8"))
```

```
signing_key = getSignatureKey(creds.secret_key, dateStamp, region, 's3')
```

```
signature = hmac.new(signing_key, stringToSign, hashlib.sha256).hexdigest()
```

```
return {  
    'statusCode': 200,  
    'headers': {  
        'Access-Control-Allow-Origin': '*',  
        'Content-Type': 'application/json'  
    },  
    'body': json.dumps({
```

```

    'stringSigned': signature,

    'stringToSign': stringToSign.decode('utf-8'),

    'xAmzCredential': creds.access_key + "/" + dateStamp + "/" + region +
    "/s3/aws4_request",

    'dateStamp': dateStamp,

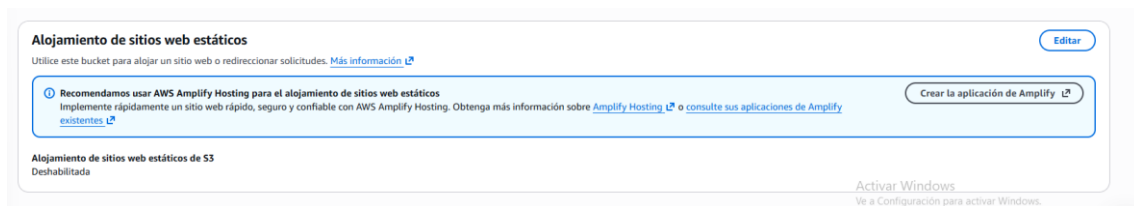
    'amzDate': amzDate,

    'securityToken': creds.token

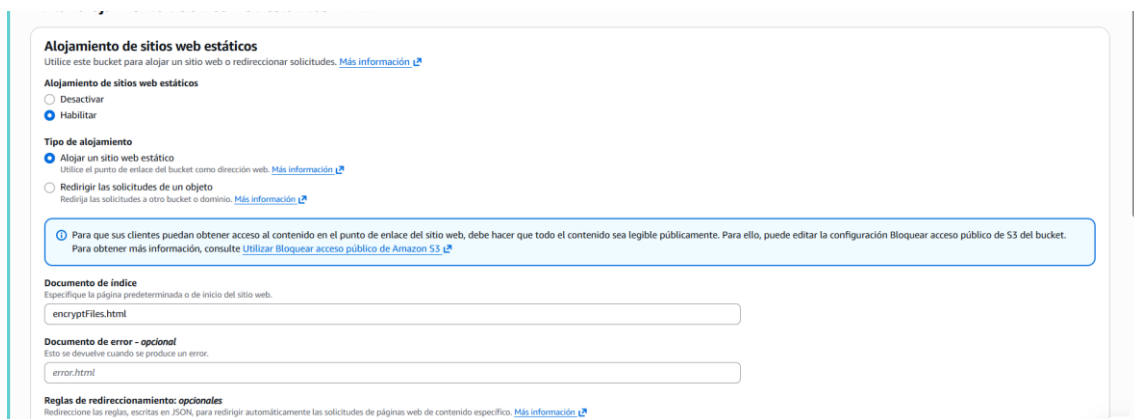
  })
}

```

una vez tenemos la función lambda (y se haya hecho deploy) y volvemos al bucket y vamos a propiedades, bajamos hasta



Lo editas y haces estos cambios:



El resto que no sale en captura, no es necesario modificarlo.

Ahora tenemos el punto de enlace

Ahora comenzamos a modificar el html

Así se ve mi enlace al bucket

```
</head>
<body onload="getAWSKeys()">

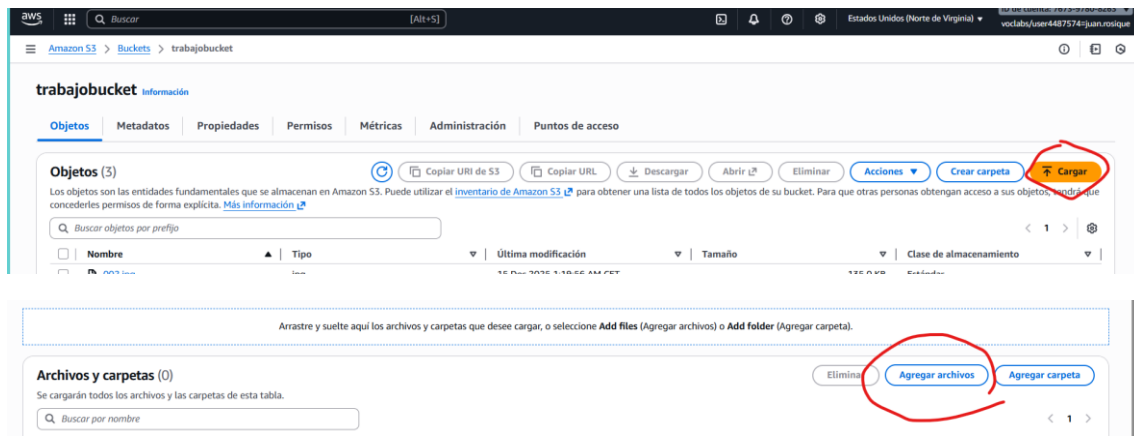
<label for="username">Username</label>
<input type="text" id="username" name="username" value="" /><br><br>

<form action="https://trabajobucket.s3.us-east-1.amazonaws.com" onsubmit="setKeyFilename()" method="post" enctype="multipart/form-data"> <!-- aqui va el bucket -->
  <input type="hidden" id="X-Amz-Credential" name="X-Amz-Credential" value="" />
  <input type="hidden" id="X-Amz-Date" name="X-Amz-Date" value="" />
  <input type="hidden" id="Policy" name="Policy" value="" />
  <input type="hidden" id="X-Amz-Signature" name="X-Amz-Signature" value="" />
  <input type="hidden" id="key" name="key" value="archivo.sln" /><br> />
  <input type="hidden" name="success_action_redirect" value="https://trabajobucket.s3.us-east-1.amazonaws.com/success.html" />
  <input type="hidden" name="X-Amz-Algorithm" value="AWS4-HMAC-SHA256" />
  <input type="hidden" id="X-Amz-Security-Token" name="X-Amz-Security-Token" value="" />
</form>
```

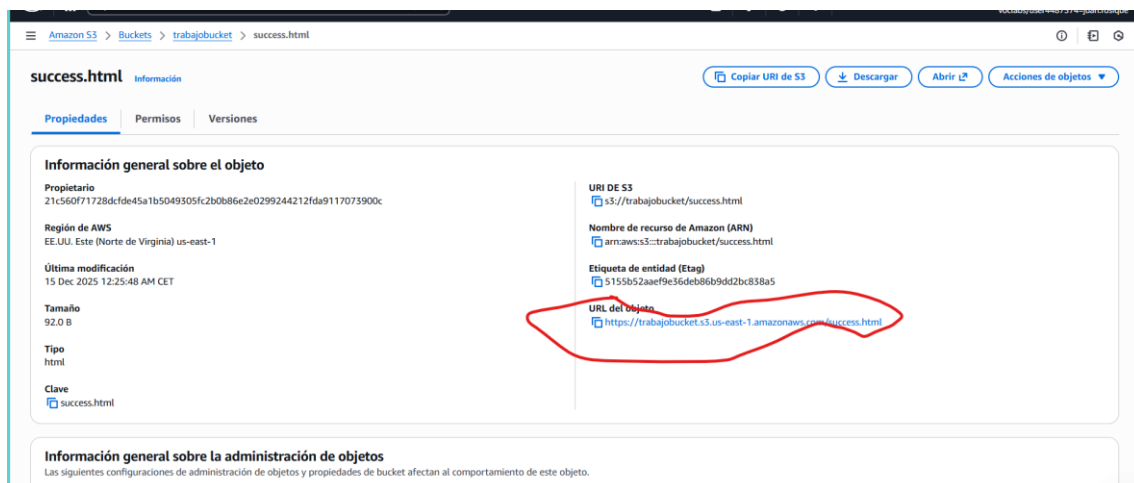
En el primer enlace subrayado va

Eseete es mi enlace al succes,

Vuelves al bucket, lo tienes que subir al bucket (success.html)



Luego pillas el enlace señalado



Este enlace es el que tienes que sustituir en el enlace de bucket del código

```
</head>
<body onload="getAWSKeys()">

<label for="username">Username</label>
<input type="text" id="username" name="username" value=""/><br><br>

<form action="https://trabajobucket.s3.us-east-1.amazonaws.com" onsubmit="setKeyfilename()" method="post" enctype="multipart/form-data"> <!--aqui va el buket-->
  <input type="hidden" id="X-Amz-Credential" name="X-Amz-Credential" value="" />
  <input type="hidden" id="X-Amz-Date" name="X-Amz-Date" value="" />
  <input type="hidden" id="Policy" name="Policy" value="" />
  <input type="hidden" id="X-Amz-Signature" name="X-Amz-Signature" value="" />
  <input type="hidden" id="key" name="key" value="fichero.sln" /><br />
  <input type="hidden" name="success_action_redirect" value="https://trabajobucket.s3.us-east-1.amazonaws.com/success.html" />
  <input type="hidden" name="X-Amz-Algorithm" value="AWS4-HMAC-SHA256" />
  <input type="hidden" id="X-Amz-Security-Token" name="X-Amz-Security-Token" value="" />
  <input type="file" name="file" />
</form>
```

en el primer enlace, pones el enlace que te viene, pero sin la direccion al html es decir

/a/b/c/d.com/e.html

Para el primer enlace se vera así->/a/b/c/d.com

Para el segundo enlace, lo dejas tal cual, con el /success.html al final

y modificar de aquí solo la parte desde la barra hasta el .s3

Ahora empezamos a cambiar el encryptfile.html, cambiando los enlaces lambda

```
function getAWSKeys() {
  var asd=$.get(
    "https://3r6pa5vht4ss6nboizgovaawyiozvadm.lambda-url.us-east-1.on.aws/", //aqui va la funcion que crea keys
  ),
  function(data) {
    var json=data;
    // json=JSON.parse(json);
    document.getElementById("Policy").value = json.stringToSign;
    document.getElementById("X-Amz-Credential").value = json.xAmzCredential;
    document.getElementById("X-Amz-Date").value = json.amzDate;
    document.getElementById("X-Amz-Signature").value = json.stringSigned;
    document.getElementById("X-Amz-Security-Token").value = json.securityToken;
  }
}

function setKeyFilename() {
  alert( "Uploading!" );

  document.getElementById("key").value = document.getElementById("file").value.substring(document.getElementById("file").value.lastIndexOf("\\")+1);
  var file=document.getElementById("key").value ;
  var user=document.getElementById("username").value;

  var sendPost=$.get(
    "https://a61e61y4dq7walvdgyfnzpzpionfakh.lambda-url.us-east-1.on.aws/", //aui va la funcion lambda basic
    {nombre: user, adjunto : file},
    function(data){
      //alert(data);
    }
  );
}
```

En el primer enlace va la segunda lambda que creamos

En el segundo enlace va el primer lambda que creamos

BUENO PUES AHORA EJECUTAMOS EN LOCAL Y LISTO

Fichero subido
